



Sort, Sweep, Mirror: Batch Private Interval Lookup with Logarithmic Cost

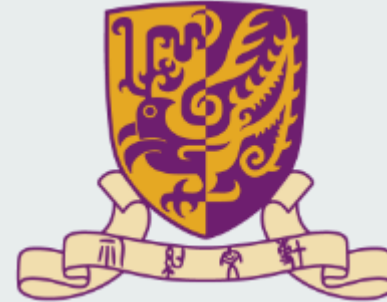
Andes Y. L. Kei¹, Lucien K. L. Ng², Jack P. K. Ma¹, Sherman S. M. Chow¹

¹ The Chinese University of Hong Kong, ² Georgia Institute of Technology

¹ {kyl022, jackma, sherman}@ie.cuhk.edu.hk, ² luciengkl@gatech.edu

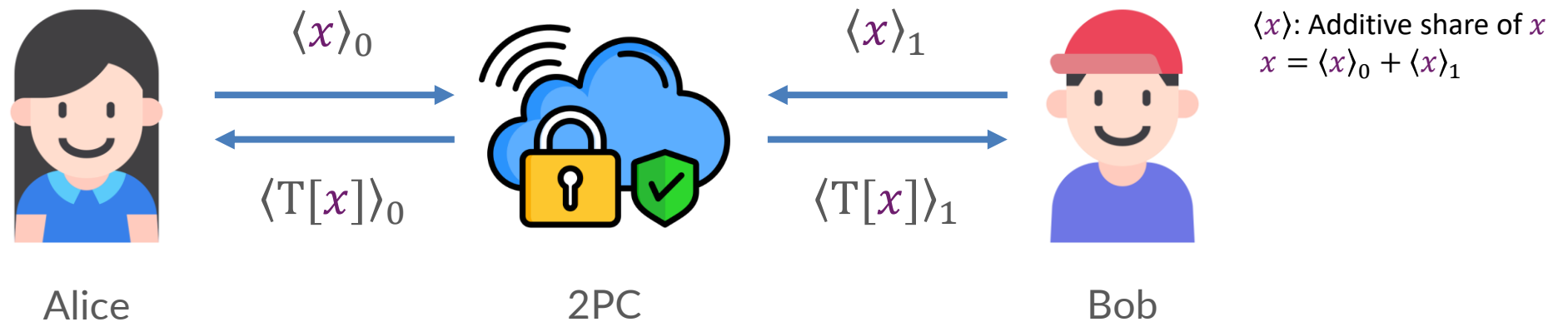


IEEE
COMPUTER
SOCIETY



Public LookUp Table (LUT) in 2-Party Computation (2PC)

- **Lookup Table:** Array T of size $L = 2^\ell$;
 - Input: **Secret-shared** index $\langle x \rangle$, $x \in \{0, 1\}^\ell$; publicly known LUT T
 - Output: **Secret-shared** value $\langle T[x] \rangle$
 - Usages: Evaluate **nonlinear** f (e.g., ReLU, GELU, e^x) via LUT of $T[x] = f(x)$ precomputed



L. Ng, S. Chow. "SoK: Cryptographic Neural-Network Computation." S&P 2023.

K. Gupta et al. "SIGMA: Secure GPT Inference with Function Secret Sharing." PETS 2024.

D. Rathee et al. "SiRNN: A Math Library for Secure RNN Inference." S&P 2021.

Interval Lookup Table (ILUT)

- Interval LUT: **Keys** are **interval boundaries** (e.g., piecewise polynomials)
 - Breakpoints k : Length- N vectors of interval starting points
 - Values v : Length- N vectors of per-interval outputs
 - Map a query q to value $v[i]$, where i is the unique index s.t. $k[i] \leq q < k[i + 1]$

- Far more compact T (typically, **# of intervals** $N \ll L$)

Equivalent size- 2^3 LUT

0	1	2	3	4	5	6	7
0	0	1	6	6	7	7	7

=

0	2	3	5
0	1	6	7

ILUT

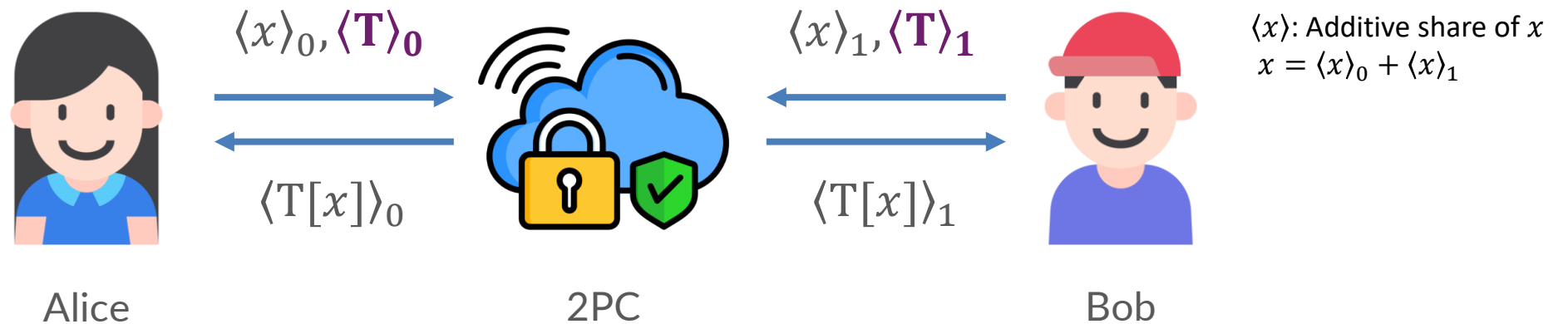
$k \rightarrow$ (points to top row of ILUT)
 $v \rightarrow$ (points to bottom row of ILUT)

$q = 2, 4, 7$ (with arrows pointing to boundaries between ILUT columns)

- Batched** Queries $q = (7, 2, 4)$ (processed at once) $\rightarrow$ Output: $T[q] = (7, 1, 6)$

Private LUT/ILUT Evaluation

- **Private LUT/ILUT: Hides the table** from both Alice and Bob
 - Input: Secret-shared index $\langle x \rangle$ and **table $\langle T \rangle$ of size $L = 2^\ell$ (LUT) or N (ILUT)**
 - Output: Secret-shared lookup value $\langle T[x] \rangle$
 - Usages: **Embedding lookup, Private decision tree evaluation, etc.**



Secure Table Lookup Schemes

Naïve ILUT expansion from LUT:
Cost blow up from N to L ($N \ll L$ typically)

Scheme	Communication	Computation	No TTP	LUT Privacy	Interval Lookup
FLUTE (S&P '23)	$\mathcal{O}(L)$	$\mathcal{O}(mL^2)$	✓		
Grotto (CCS '23)	$\mathcal{O}(\log L)$	$\mathcal{O}(N \log L)$			✓
logrow (Eurocrypt '24)	$\mathcal{O}(mL)$	$\mathcal{O}(mL)$	✓	✓	
Lu <i>et al.</i> (NDSS '25)	$\mathcal{O}(L \log L)$	$\mathcal{O}(L^2)$	✓	✓	
Ours					

- **Linear complexity barrier** due to independent query processing

- Communication: $\Omega(mL)$ or require a **trusted third party (TTP)**
- Computation: $\Omega(mL)$ or $\Omega(N \log L)$ **even with a TTP**

ℓ : Input bitwidth, $L = 2^\ell$
 m : Output bitwidth
 N : Number of intervals

A. Brüggemann et al. "FLUTE: Fast and Secure Lookup Table Evaluations." *S&P* 2023.

K. Storrier et al. "Grotto: Screaming Fast $(2 + 1)$ -PC for $\mathbb{Z}_{2^n}$ via $(2, 2)$ -DPFs." *CCS* 2023.

D. Heath, V. Kolesnikov, L. Ng. "Garbled Circuit Lookup Tables with Logarithmic Number of Ciphertexts." *Eurocrypt* 2024.

T. Lu et al. "A New PPML Paradigm for Quantized Models." *NDSS* 2025.

Secure Table Lookup Schemes

Scheme	Communication	Computation	No TTP	LUT Privacy	Interval Lookup
FLUTE (S&P '23)	$\mathcal{O}(\textcolor{red}{L})$	$\mathcal{O}(m\textcolor{red}{L}^2)$	✓		
Grotto (CCS '23)	$\mathcal{O}(\log L)$	$\mathcal{O}(\textcolor{red}{N} \log L)$			✓
logrow (Eurocrypt '24)	$\mathcal{O}(m\textcolor{red}{L})$	$\mathcal{O}(m\textcolor{red}{L})$	✓	✓	
Lu <i>et al.</i> (NDSS '25)	$\mathcal{O}(\textcolor{red}{L} \log L)$	$\mathcal{O}(\textcolor{red}{L}^2)$	✓	✓	
Ours	$\mathcal{O}(\ell \textcolor{violet}{\log N})$	$\mathcal{O}((\ell + m) \textcolor{violet}{\log N})$	✓	✓	✓

- “Sort, sweep, mirror” **breaks** the **barrier** via amortization
 - Offers **batch** private ILUT without a TTP
 - Realizes **logarithmic** communication and computation
 - Takes **1984×** communication reduction for size- 2^{24} LUT vs. logrow

ℓ : Input bitwidth, $\textcolor{red}{L} = 2^\ell$
 m : Output bitwidth
 $\textcolor{violet}{N}$: Number of intervals

A. Brüggemann et al. “FLUTE: Fast and Secure Lookup Table Evaluations.” S&P 2023.

K. Storrier et al. “Grotto: Screaming Fast $(2 + 1)$ -PC for $\mathbb{Z}_2^n$ via $(2, 2)$ -DPFs.” CCS 2023.

D. Heath, V. Kolesnikov, L. Ng. “Garbled Circuit Lookup Tables with Logarithmic Number of Ciphertexts.” Eurocrypt 2024.

T. Lu et al. “A New PPML Paradigm for Quantized Models.” NDSS 2025.

Secure Table Lookup Schemes

Scheme	Communication	Computation	No TTP	LUT Privacy	Interval Lookup
Ours	$\mathcal{O}(\ell \log N)$	$\mathcal{O}((\ell + m) \log N)$	✓	✓	✓

- Batch queries are *natural*: Millions of nonlinear function calls in transformers
- Concurrent Work:
 - FABLE (Usenix Sec. '25): **Batch** LUT, **relaxed** privacy (one party knows the table)
 - Toss (CCS '25): **Public** LUT, $\mathcal{O}(mL^{1/2})$ communication
 - MARLUT (S&P '26): **Public** LUT, **malicious 3PC**, $\mathcal{O}(L^{1/3})$ communication

Z. Su et al. "FABLE: Batched Evaluation on Confidential Lookup Tables in 2PC." *Usenix Security* 2025.

L. Ng, V. Kolesnikov. "Toss: Garbled PIR from Table-Only Stacking." *CCS* 2025.

C. Li et al. "Secure Lookup Tables: Faster, Leaner, and More General." *S&P* 2026.

“Sort” *keys and queries* together

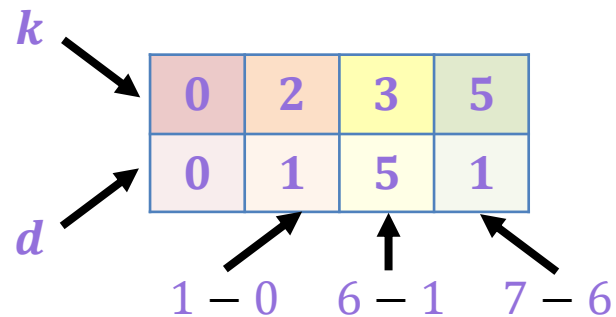
“Sweep” via *prefix sum*

“Mirror” *results* back

*amortized logarithmic cost
for batch private table lookup!*

Our Three-Phase **Batch** Lookup Approach

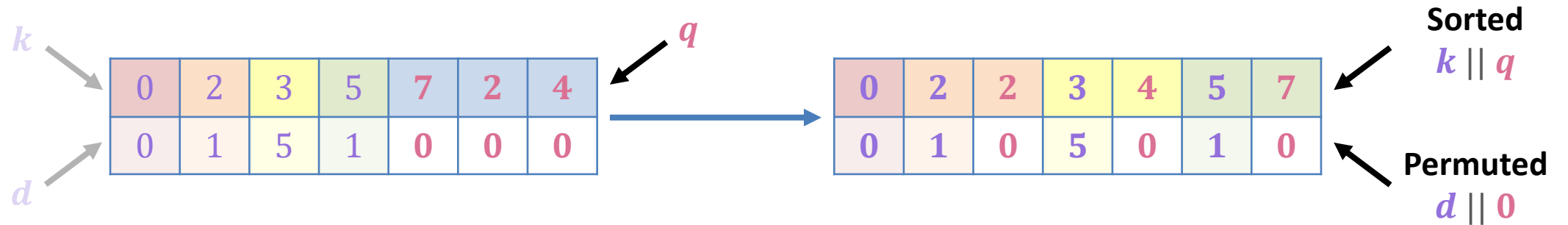
- Secret-shared Inputs: $k = (0, 2, 3, 5)$, $v = (0, 1, 6, 7)$
- *One-Time* Preprocessing: Computes differences of values $d[i] = v[i] - v[i - 1]$



- **Why?** Prefix sum then “reconstructs” $T[x]$
- **Cost:** (Cheap, non-interactive) Secret-shared difference

Our Three-Phase **Batch** Lookup Approach

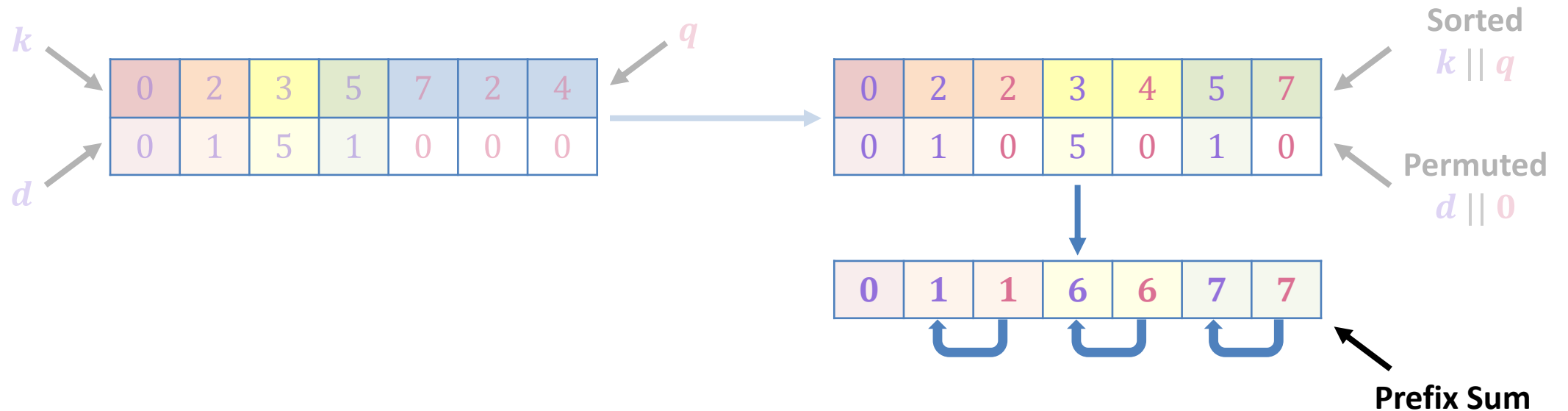
- **Sort** (obliviously) the merged list of table keys k and queries q
 - $k = (0, 2, 3, 5), v = (0, 1, 6, 7), q = (7, 2, 4) \rightarrow T[q] = (7, 1, 6)$ (all secret-shared)



- After sorting, each query lands immediately after its interval's start key
- **Cost:** Oblivious (*i.e.*, access-pattern-concealing) sort of $\mathcal{O}(N)$ items

Our Three-Phase **Batch** Lookup Approach

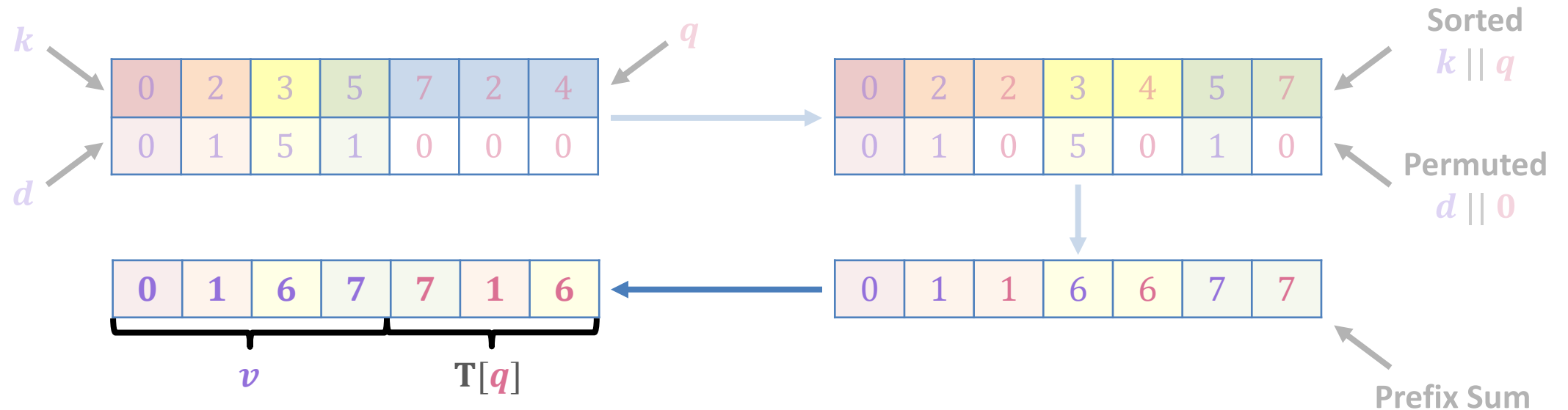
- **Sweep:** Computes the **prefix sum** on permuted $d \parallel 0$
 - $k = (0, 2, 3, 5, 7, 2, 4)$, $v = (0, 1, 6, 7)$, $q = (7, 2, 4) \rightarrow T[q] = (7, 1, 6)$ (all secret-shared)



- **Cost:** (Cheap, non-interactive) Secret-shared prefix sum

Our Three-Phase **Batch** Lookup Approach

- **Mirror:** Reverts to original query order
 - $k = (0, 2, 3, 5), v = (0, 1, 6, 7), q = (7, 2, 4) \rightarrow T[q] = (7, 1, 6)$ (all secret-shared)



- **Cost:** Secret-shared shuffle (much cheaper than oblivious sort)



Sort, Sweep, Mirror: Summary

- Primary cost of processing $\mathcal{O}(N)$ queries is a **single oblivious sort of $\mathcal{O}(N)$ items**
 - $\mathcal{O}(N \log N)$ secure comparison, *i.e.*, **$\mathcal{O}(\log N)$** per-query
- More in Our Paper:
 - **Efficiently “un-do” the sort?** “Shuffle-then-sort” paradigm (CCS '21)
 - **What if # of queries $\gg$ # of intervals N ?** Query partitioning
 - **Constant-round lookup?** Alon—Azar—Vishkin sort (FOCS '86) + garbled circuits

Final Remarks

- **First batch private interval lookup**

- Achieving **logarithmic** communication and computation cost in table size
- Reducing **1984x** communication for size- 2^{24} LUT over the state of the art



<https://doi.ieeecomputersociety.org/10.1109/SP63933.2026.00113>

- Future Directions:

- Optimize online cost by **preprocessing** (AHE/DPF-based comparison?)
- Improve overall efficiency by **relaxing privacy requirement** (differential privacy?)
- Application to **private NN computation** (ILUT-based inference on transformers/quantized NNs?)

- Contact: {[kyl022](#), jackma, sherman}@[ie.cuhk.edu.hk](#), [luciengkl@gatech.edu](#)

L. Ng, S. Chow. "GForce: GPU-Friendly Oblivious and Rapid Neural Network Inference." *Usenix Security* 2021.

H. Saleem et al. "Hawk: Accurate and Fast Privacy-Preserving Machine Learning Using Secure Lookup Table Computation." *PETS* 2024.

K. Gupta et al. "SIGMA: Secure GPT Inference with Function Secret Sharing." *PETS* 2024.